Stem Agent: a report

Krzysztof Kopel

1. My approach

When I read the task description for the first time, I thought that the description of a stem agent is quite similar to how humans think and work. We all have some skillsets and choose a skill from that set, based on the task specified. So that is what my stem model is trying to achieve at the beginning – identify the sort of task it needs to complete and choose appropriate methods from its predefined set of actions. The general path of evolution in my model goes as follows: first we have a generic stem agent, which identifies the problem, and based on that identification, it creates a specialized agent, which has a smaller (more precise) list of actions available. The specialized agent can (but does not have to) use other findings of the basic agent to produce the final solution to the problem specified.

Then came the selection of the exemplary topic: I decided to try to focus on a networking problem. We have a failure that manifests itself on OSI layer 7 (application), with errors from technologies like gRPC and RabbitMQ. A naïve model would think that solving these problems comes down to changing how the two technologies are configured, but in my case, the error resides somewhere else. The routing algorithm (I chose OSPF) couldn't establish routing tables, because in the configuration of one of the routers along the way someone set the MTU (maximum transmission unit) to 1400 bytes, which is too low for some of that protocol's messages to come through (hello packets are fine, but database descriptions (DBD) are too large, so they get dropped). Because of that, OSPF stays in the EXSTART state, while it should proceed to the FULL state. So, we have a problem which requires looking at different layers of the networking stack and noticing that there are particular actions taken at layer 3 that influence how layer 7 software behaves.

Another thing that I tried to focus on was the output format: conventional LLMs usually produce a wall of text, which uses up tokens, while the answer itself is just a small fraction of what was generated. I decided to go for a structured approach: the "evolved" agent first lists the actions to perform and only thereafter it provides some sort of rationalization, telling the user why the model wants to perform these particular actions. Additional advantage of that approach is that placing the executable payload at the top of the Pydantic schema allows for immediate validation, treating the reasoning section as supplementary telemetry rather than a parsing bottleneck.

2. Architecture

As I mentioned, there are two main model schemes defined:

a. Stem model (stem_model.py) which is the generic one, responsible for checking which tools might be useful for our task, defining the testing hypothesis and formulating conditions necessary for a success.
b. Specialized model (specialized_model.py) which gets the list of available tools and the findings of the previous model and generates the result using these tools.

The tools come from a closed catalogue (tools.py) which can be expanded by the user if needed, in order to handle different tasks. The formats for answers from the OpenAI API was defined using Pydantic. I used gpt-4o model, I explain my choice in the chapter regarding experiments.

For the evolution of the agent I used logs from Cisco IOS on a router (showing commands provided by a user and OSPF state), RabbitMQ and gRPC. These files are present in the resources directory. The logs are not real (they are AI-generated and human-checked), but they conform to the standards set by the protocols and technologies described, and are sufficient for the problem described in the first chapter.

The complete technological demo of my solution is available in the demo.ipynb file (and I really recommend checking it out).

3. Experiments and failures

a. The choice of a model

My initial intuition was to use the newest model, one from the gpt-5 family, because I thought that was the best way to showcase the potential of my architecture. Although when I read a little bit more, I discovered that it introduces a new Responses API, which I am not familiar with (and because of the time constraints I couldn't spend time getting to know it, although I am definitely going to do that in near future). Additionally, the newer models are more expensive (in terms of price per token), while the slightly older gpt-4 family is perfectly fine for the task. Consequently I went for the gpt-4o model, which turned out to be sufficient. Its highly stable structured output capabilities paired perfectly with the Pydantic validation pipeline, proving entirely sufficient and highly resource-efficient for layer-3 to layer-7 cross-analysis.

b. Initial prompt

The hardest part was the hypothesis field in one of response structures. I wanted it to be a sort of "intellectual bridge" between the stem and advanced agents. I needed the model to focus strictly on causality, without any vague or otherwise unnecessary references and at first the responses in this field were unsatisfactory. The solution that I found was actually quite simple, because Pydantic already has a built in Field class that allows to provide a description for a specific field in the response. The other fields behaved as I expected them to, and writing an appropriate prompt was a problem.

c. Pydantic limitations

Another issue that I encountered was the fact that Pydantic doesn't support all of Python data types (I had a problem with tuples), but I found a way around by creating a separate class where otherwise I would use a tuple.

4. What would I do if I had more time?

- Add more tools to the already present ones, since they were designed to match the networking needs (although it can be seen in my demo that not all of them were used, so the model actually differentiated as needed).
- Experiment with different LLMs. I wonder how would the agents behave if I used Google Gemini or Claude instead of solutions from OpenAI.
- Broaden the scope of the advanced model – currently the structure returned by the advanced model uses tool-command pairs, which was natural for my task connected with networking and is natural for many computer science-related problems, but would probably be insufficient for many other areas.
- Add (at least) one more step to the evolution process – at the moment a stem models transforms into an advanced one, but it is not hard to imagine a situation where we would want or need a more complicated architecture.